CPU Scheduling Simulator and Performance Analysis

Operating Systems Project Report:
CPU Scheduling Simulator and Performance Analysis

Prepared By:

- Basema Thabet Odat : 202310822
- Diala Belal Zaki Awwad : 202311057
- Ghufran Marai Dorgham : 202311523
- Sara Luay Ibrahim Bataineh : 202311093
- Sedra Bashar Emad Addin Alsaqqal : 20230832

Course:**Operating Systems**
Instructor:**Dr.Mohammad Alomar**
University: **Irbid National University**

1. Introduction

Modern operating systems are responsible for managing hardware resources efficiently while ensuring that multiple processes can execute smoothly and fairly. One of the most critical responsibilities of the operating system is CPU scheduling. CPU scheduling determines which process will use the CPU at a specific moment and directly affects the overall system performance, responsiveness, throughput, and user experience.
In multitasking environments, many processes compete simultaneously for CPU access. Without an efficient scheduling mechanism, the CPU may become inefficiently utilized, processes may experience excessive waiting times, and system responsiveness may degrade significantly. For this reason, operating systems implement different scheduling algorithms, where each algorithm follows a unique strategy for process selection and execution.
This project focuses on designing and implementing a complete CPU Scheduling Simulator using Python. The simulator implements four fundamental scheduling algorithms:

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Round Robin (RR)
- Priority Scheduling

The system evaluates the behavior and efficiency of each scheduling algorithm under multiple workload scenarios. Additionally, the simulator generates detailed Gantt charts and performance comparison graphs to visualize execution order and analyze scheduling efficiency.
The primary purpose of this project is not only to implement scheduling algorithms but also to perform analytical comparisons between them using important performance metrics such as:

- Average Waiting Time
- Average Turnaround Time
- Fairness
- Responsiveness
- Scheduling Efficiency

The project demonstrates how different scheduling approaches behave under various workloads and highlights the strengths and limitations of each algorithm in practical operating system environments.

2. Project Objectives

The main objectives of this project are:

- To design and implement a CPU Scheduling Simulator using Python.
- To simulate the execution behavior of multiple scheduling algorithms.
- To calculate scheduling performance metrics accurately.
- To generate graphical Gantt chart visualizations for process execution.
- To compare algorithm performance under different scheduling scenarios.
- To analyze the efficiency, fairness, and responsiveness of each scheduling technique.
- To identify which algorithms perform better under specific workload conditions.
- To provide practical understanding of CPU scheduling concepts in operating systems.

3. CPU Scheduling Algorithms

3.1 First Come First Serve (FCFS)

First Come First Serve is the simplest CPU scheduling algorithm. Processes are executed according to their arrival order in the ready queue. The process that arrives first receives CPU execution first.

Working Principle

- Processes are inserted into the ready queue according to arrival time.
- The scheduler selects the earliest arriving process.
- Once a process starts execution, it continues until completion.

Advantages

- Simple and easy to implement.
- Fair in terms of arrival order.
- Minimal scheduling overhead.

Disadvantages

- Poor average waiting time.
- Suffers from the convoy effect.

- Long processes can delay short processes significantly.
- Weak responsiveness for interactive systems.

Suitable Environments
FCFS is suitable for simple batch systems where fairness is more important than responsiveness.

3.2 Shortest Job First (SJF)
Shortest Job First selects the process with the smallest CPU burst time.

Working Principle
- Among all ready processes, the scheduler selects the process with the shortest burst time.
- Short processes complete earlier, reducing average waiting time.

Advantages
- Produces optimal average waiting time in many cases.
- Improves system throughput.
- Efficient for systems containing many short tasks.

Disadvantages
- Possible starvation for long processes.
- Burst time prediction may not always be accurate.
- More complex than FCFS.

Suitable Environments
SJF is highly efficient in systems where burst times can be estimated accurately.

3.3 Round Robin (RR)
Round Robin scheduling allocates CPU time using fixed time slices called quantum.

Working Principle
- Each process receives CPU execution for a limited time quantum.
- If execution is incomplete, the process returns to the end of the ready queue.
- CPU cycles continuously between processes.

Advantages
- High fairness.
- Excellent responsiveness.
- Suitable for time-sharing systems.
- Prevents process starvation.

Disadvantages

- Performance depends heavily on quantum size.
- Excessive context switching may reduce efficiency.
- Larger turnaround times in some workloads.

Suitable Environments
Round Robin is ideal for interactive and multi-user operating systems.

3.4 Priority Scheduling
Priority Scheduling executes processes according to priority values.

Working Principle
- Each process is assigned a priority.
- Higher priority processes execute before lower priority processes.
- Equal priorities may follow FCFS ordering.

Advantages
- Important processes receive faster execution.
- Flexible scheduling behavior.
- Useful for real-time and critical systems.

Disadvantages
- Starvation may occur for low-priority processes.
- Requires careful priority management.
- Fairness may decrease significantly.

Suitable Environments
Priority Scheduling is commonly used in systems containing critical or time-sensitive processes.

4. System Design and Methodology
The simulator was designed to emulate realistic CPU scheduling behavior while maintaining modularity and scalability.
The system accepts multiple process inputs where each process contains:
- Process ID
- Arrival Time
- Burst Time
- Priority

The simulator executes each scheduling algorithm independently and calculates scheduling statistics automatically.

4.1 Performance Metrics

Waiting Time (WT)

Waiting Time represents the amount of time a process spends waiting in the ready queue before execution.

WT = TAT - BT

Where:

- WT = Waiting Time
- TAT = Turnaround Time
- BT = Burst Time

Turnaround Time (TAT)

Turnaround Time measures total completion duration from process arrival until execution completion.

TAT = CT - AT

Where:

- CT = Completion Time
- AT = Arrival Time

4.2 Gantt Chart Visualization

The simulator generates graphical Gantt charts using Matplotlib to visualize:

- Process execution order
- CPU allocation timeline
- Scheduling behavior differences

These visualizations help analyze fairness, process switching frequency, and scheduling efficiency.

5. Implementation

The project was implemented using Python due to its simplicity, flexibility, and strong visualization libraries.

Technologies Used

- Python 3
- Matplotlib
- Pandas

Main Functionalities

The simulator performs the following operations:

- Process scheduling simulation
- Waiting time calculation
- Turnaround time calculation
- Performance comparison
- Gantt chart generation
- Scenario-based workload testing

Program Structure

The implementation consists of several independent scheduling functions:

- fcfs_scheduling()
- sjf_scheduling()
- round_robin_scheduling()
- priority_scheduling()

Additional functions were created for:

- Visualization
- Statistics calculation
- Comparative analysis

This modular structure improves readability, maintainability, and scalability.

6. Experimental Scenarios

To evaluate scheduling behavior accurately, multiple workload scenarios were tested.

6.1 Balanced Workload Scenario

This scenario contains processes with relatively balanced burst times and arrival times.

Observations

- SJF produced lower waiting times.
- Round Robin achieved better fairness.
- FCFS performed adequately but showed larger waiting times for short processes.

6.2 Stress Test Scenario

This scenario simulated heavy CPU workload with many processes and long burst times.

Observations

- Round Robin generated excessive context switching.
- FCFS suffered from large delays due to long processes.
- SJF maintained superior performance efficiency.
- Priority Scheduling favored high-priority tasks effectively.

6.3 Starvation-Oriented Scenario

This scenario focused on workload imbalance and priority conflicts.

Observations

- Priority Scheduling risked starving low-priority processes.
- SJF delayed large processes significantly.
- Round Robin maintained fairness despite slightly larger turnaround times.

7. Results and Analysis

The generated results demonstrated significant behavioral differences between scheduling algorithms.

FCFS Analysis

FCFS produced larger waiting and turnaround times because processes execute strictly according to arrival order. Long burst processes delay all subsequent processes, causing the convoy effect.
The Gantt charts clearly show continuous execution blocks with minimal interruption, which reduces context switching but decreases responsiveness.

SJF Analysis

SJF consistently achieved the best average waiting time and turnaround time across most scenarios.
The algorithm prioritizes shorter tasks, allowing rapid process completion and improved throughput. However, longer processes may wait excessively, especially in workloads containing many short tasks.
The charts demonstrate more optimized CPU allocation compared to FCFS.

Round Robin Analysis

Round Robin achieved the highest fairness and responsiveness.
The repeated CPU rotation prevented starvation and ensured all processes received execution opportunities regularly. However, excessive context switching increased scheduling overhead.
The Gantt charts clearly reveal frequent process switching patterns caused by quantum expiration.

Priority Scheduling Analysis

Priority Scheduling effectively handled critical processes by executing higher-priority tasks earlier.
The algorithm demonstrated good performance under workloads containing important tasks. However, low-priority processes experienced increased waiting times in some scenarios.
The results indicate that Priority Scheduling balances efficiency and responsiveness when priorities are assigned properly.

8. Comparative Discussion

The performance comparison graphs reveal that no single scheduling algorithm is universally optimal.

- SJF achieved the lowest average waiting and turnaround times.
- Round Robin provided the best fairness and responsiveness.
- FCFS remained the simplest implementation.

- Priority Scheduling effectively handled critical tasks.

The best scheduling algorithm depends heavily on workload characteristics and system requirements.
Interactive systems benefit more from Round Robin, while batch processing systems may benefit more from SJF.

9. Conclusion

This project successfully implemented and analyzed four major CPU scheduling algorithms using Python.
The simulator demonstrated how scheduling strategies directly affect operating system performance and process behavior. Through graphical visualization and statistical analysis, the project highlighted the strengths and weaknesses of each scheduling technique.
Experimental results showed that:

- SJF provides the best average performance metrics.
- Round Robin offers superior fairness and responsiveness.
- FCFS is simple but inefficient under heavy workloads.
- Priority Scheduling is highly effective for critical process handling.

Overall, CPU scheduling remains one of the most important optimization mechanisms in operating systems, and selecting the appropriate scheduling algorithm depends on workload type, responsiveness requirements, and system objectives.

10. Future Improvements

Future development may include:

- Multilevel Queue Scheduling
- Multilevel Feedback Queue
- Real-Time Scheduling Algorithms
- GUI-Based Interactive Simulator
- Dynamic Quantum Adjustment
- Live Process Injection
- CPU Utilization Analysis
- Memory Management Integration

These improvements would further increase realism and simulation complexity.

References

1. Abraham Silberschatz, Peter B. Galvin, Greg Gagne — *Operating System Concepts*
2. William Stallings — *Operating Systems: Internals and Design Principles*
3. Python Documentation — https://www.python.org
4. Matplotlib Documentation — https://matplotlib.org